



Test Technique

Projet : ClinicFlow – Gestion Patients & Rendez-vous

Objectif

Construire une application web (PERN) pour une petite clinique afin de gérer :

- **patients**
- **rendez-vous**
- **utilisateurs**
- un **dashboard simple**

Tout en respectant une conception DB solide + API propre + UI fonctionnelle.

1) Livrables attendus

Le candidat doit fournir :

1. Conception (obligatoire)

- Schéma base de données (ERD) ou document clair
- Justification rapide des relations (1-1, 1-N, N-N)

2. **Backend API (Node + Express)**
 3. **Frontend (React)**
 4. **Script DB / migrations PostgreSQL**
 5. **README** (installation + env + commandes)
-

2) Fonctionnalités obligatoires

2.1 Authentification & rôles

Rôles

- `admin`
- `staff`

Obligatoire

- Login par **email + password**
 - Auth via **JWT**
 - Middleware `auth`
 - Middleware `role` (admin-only pour certaines actions)
-

2.2 Gestion des Patients

Champs minimum

- `fullName`
- `CIN` (unique)
- `phone`
- `birthDate`
- `address` (optionnel)
- `createdAt`

Actions API

- Créer patient
 - Modifier patient
 - Supprimer patient (**admin only**)
 - Liste paginée + recherche `fullName` / `CIN`
-

2.3 Gestion des Rendez-vous

Champs minimum

- `patientId`
- `appointmentDate` (date + heure)
- `status` : `pending` , `confirmed` , `cancelled`
- `reason` (motif)
- `notes` (optionnel)
- `createdBy` (userId)

Règle métier obligatoire

Un patient ne peut pas avoir **2 rendez-vous confirmés dans une fenêtre de 30 minutes**.

2.4 Dashboard (statistiques)

Afficher :

- Total patients
 - Rendez-vous du jour
 - Nombre `pending`
 - Nombre `confirmed`
-

3) Partie Conception (obligatoire)

Le candidat doit **concevoir la DB avant le code**.

Tables minimales attendues

- `users`
- `patients`
- `appointments`

Contraintes attendues

- UUID PK
- `patients.cin` UNIQUE
- FK + cascade rules (au choix, mais justifiées)
- Index sur les colonnes de recherche

📌 Bonus conception (facultatif)

- `deleted_at` (soft delete)
 - audit log table
 - ENUM Postgres ou CHECK constraints
-

4) API REST minimale attendue

Auth

- `POST /api/auth/login`
- `GET /api/auth/me`

Patients

- `POST /api/patients`
- `GET /api/patients?search=&page=&limit=`
- `GET /api/patients/:id`
- `PUT /api/patients/:id`
- `DELETE /api/patients/:id` (admin)

Appointments

- `POST /api/appointments`
- `GET /api/appointments?date=&status=`

- PATCH /api/appointments/:id/status
-

5) Frontend React (obligatoire)

Pages minimum

1. Login

2. Dashboard

3. Patients list

- recherche + pagination
- bouton add/edit

4. Patient details

- infos patient
- liste de ses rendez-vous

5. Appointments

- création rendez-vous
 - filtres (date/status)
 - update status
-

6) Contraintes de qualité

Obligatoire :

- Validation input (Zod/Joi)
- Password hash (bcrypt)
- Gestion erreurs propre (400 / 401 / 403 / 404 / 500)
- Code structuré (routes/controllers/services)

Bonus (facultatif)

- Docker compose (frontend + backend + postgres)
- Swagger / OpenAPI
- Tests (Jest / Supertest)

7) Données de test (seed)

Le candidat doit fournir un script ou migration avec :

- 1 admin
 - 2 staff
 - 5 patients
 - 10 rendez-vous (mix pending/confirmed/cancelled)
-

8) Critères d'évaluation

On évalue :

- qualité de la **conception DB**
- respect des règles métiers
- structure backend (propre et scalable)
- UI fonctionnelle
- sécurité JWT + hash password
- performance (index, pagination)